

Telas finais apresentadas do Simulador de SO Multitarefa - Parte B

João Vitor Bernardi RA: 2699559

Lais Sayoko Toda Duarte RA:2699567

gitHub para repositório:

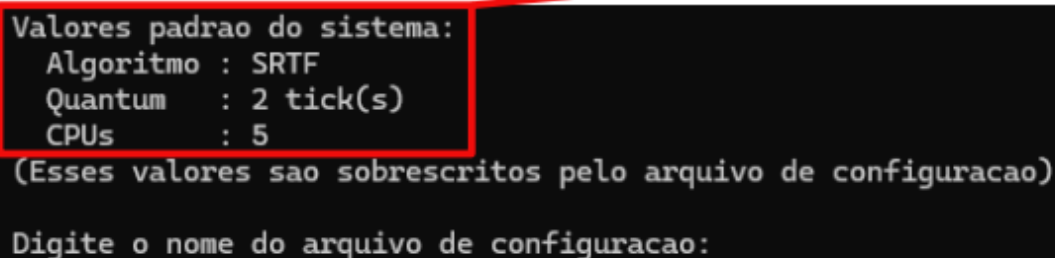
<https://github.com/joaobernardi2006/Simulador-de-SistemaOperacional-Multitareas.git>

Após a finalização da implementação de todas as funções e requisitos solicitados para o Simulador de um Sistema Operacional Multitarefa, temos diversos elementos visuais, que são apresentados tanto no terminal durante a execução da simulação, quanto ao fim da mesma, no arquivo de tipo **.svg**.

1. Telas apresentadas no Terminal

1.1. Tela de definição do arquivo par a simulação

Valores
padrões



```
Valores padrao do sistema:  
  Algoritmo : SRTF  
  Quantum   : 2 tick(s)  
  CPUs      : 5  
(Esses valores sao sobrescritos pelo arquivo de configuracao)  
  
Digite o nome do arquivo de configuracao:
```

Nessa primeira tela que aparece logo após inicializar a simulação, é solicitado com que o usuário escolha o arquivo **.txt** do qual será utilizado para a ocorrência da simulação e consequentemente da formação do diagrama de Gantt. O arquivo necessariamente precisa estar no mesmo diretório que os arquivos de execução do simulador e seu formato deve seguir o padrão já explicitado no documento de requisitos:

algoritmo_escalamento;quantum;qtde_cpus;alpha

id;cor;ingresso;duracao;prioridade;lista_eventos

Assim, temos neste arquivo o algoritmo de escalonamento que será utilizado pelo simulador (SRTF, PRIOP ou PRIOPEnv), o quantum para as tarefas, a quantidade de CPUs que a simulação deve trabalhar e, no caso do PRIOPEnv, o alpha (fator de envelhecimento). Nas linhas seguintes, temos o id da tarefa, bem como sua cor, ingresso, duração, prioridade de execução (essencial para o PRIOP e o PRIOPEnv) e a lista de eventos, respectivamente. Vale destacar que, no Projeto B, a lista de eventos passa a incluir as operações de mutex (ML/MU) e de entrada/saída (IO), tratando a sincronização entre tarefas e os bloqueios por E/S.

Temos também no terminal os valores padrões que já foram definidos inicialmente pelo próprio sistema. Esses elementos padrões derivam da função "**inicializaConfigSistema()**", que os define. Agora, a leitura do arquivo é feita pela função "**lerConfiguracao()**".

1.2. Tela de escolha do modo de execução

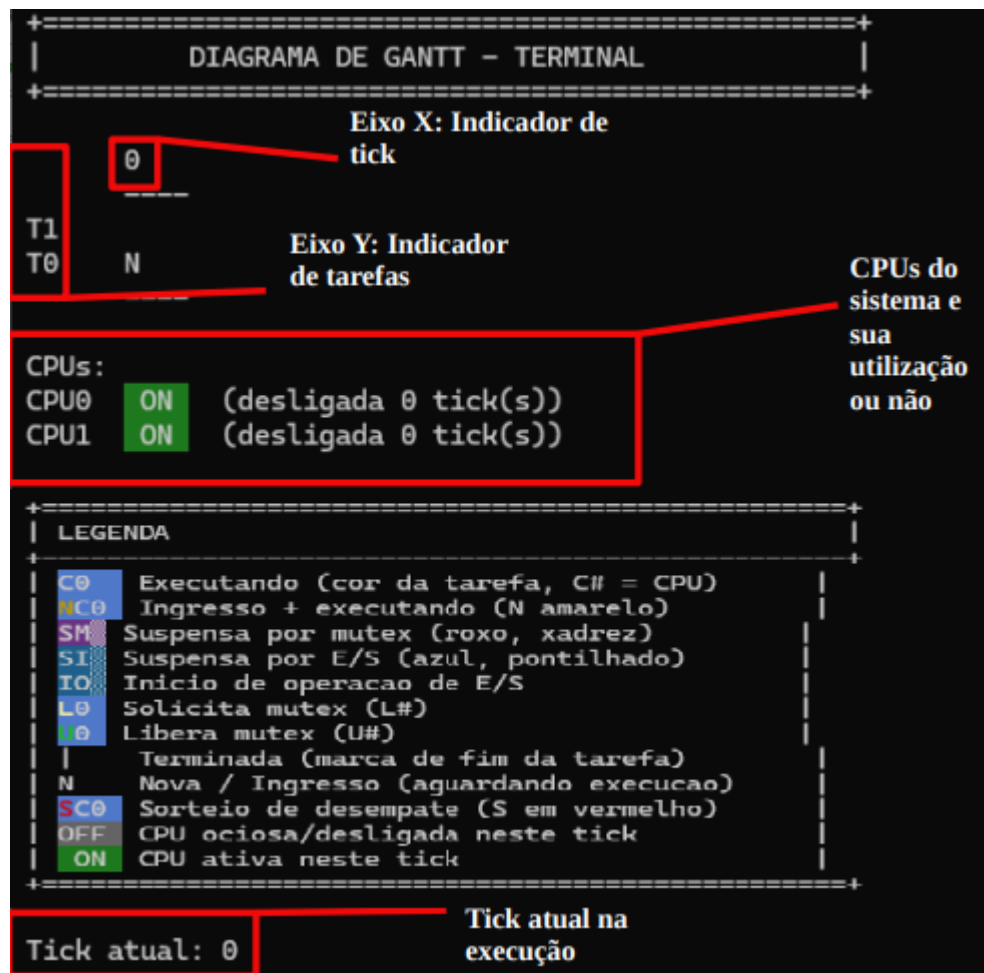
```
Valores padrao do sistema:
  Algoritmo : SRTF
  Quantum   : 2 tick(s)
  CPUs      : 5
(Esses valores sao sobrescritos pelo arquivo de configuracao)

Digite o nome do arquivo de configuracao: teste.txt
Como deseja realizar a simulacao? Passo-a-passo(a) ou completa(b)?
```

Após essa escolha do arquivo que define as informações do simulador, temos agora uma mensagem que busca identificar qual é o modo de execução a ser seguido, podendo ser passo a passo (semelhantemente a um debugger) e a execução completa (A simulação é realizada até o final, sem interrupções). Caso o usuário digite a letra "A", o simulador entende que o modo de execução selecionado é o passo a passo, e caso o usuário digite "B", temos a escolha da execução completa. Independentemente do usuário digitar letra minúscula ou maiúscula, é realizado o tratamento para garantir com que seja algo adequado. Se for digitado um caractere aleatório, é exibido um erro. Essa escolha para saber qual deve ser o modo de execução é feita diretamente da função "**main()**", onde caso seja optado por seguir a opção Passo-a-Passo, é feito uma chamada para a função "**execucaoPassoAPasso()**", caso seja escolhido a opção de execução completa, a função chamada é "**execucaoCompleta()**".

```
Opcao invalida!
Process returned 1 (0x1)   execution time : 5.538 s
Press any key to continue.
```

1.3. Execução Passo a Passo



Ao optar pela execução passo a passo (a), o usuário depara-se com esta tela, onde temos a estrutura base do diagrama de Gantt. Ao topo, temos o título do diagrama e logo abaixo, no eixo Y, temos a indicação das tarefas ordenadas de forma decrescente por ID (de baixo para cima). O eixo X indica a evolução dos ticks. Abaixo do diagrama, temos um indicador para quando um determinado CPUx está ligado (ON) ou não (OFF) e, ao seu lado, um contador que indica a quantidade de ticks em que aquela CPU permaneceu desligada. Por fim, temos uma legenda que indica o que cada símbolo apresentado no diagrama representa, além do tick atual no qual o simulador se encontra.

O diagrama gerado no terminal é feito por meio de "***imprimir_gantt_terminal()***", utilizando majoritariamente a variável ***DiagramaGantt **diagrama***, matriz que armazena, para cada tarefa e tick, o estado da tarefa, a CPU usada, se foi sorteada ou não e como novidade do Projeto Bo e, vento de mutex ocorrido (ML/MU), o número do mutex e o motivo da suspensão (por mutex ou por E/S).

Por sua vez, os estados das tarefas são definidos por meio de ***#define***, e cada estado particular é armazenado pela variável ***int estado***, presente na struct ***TCB***. Nessa mesma struct temos armazenado: ID, estado, duração, prioridade, tempo restante, tempo de espera, cor, quantum e a lista de eventos (agora com as ações de mutex e E/S). Ademais, as informações referentes às CPUs em uso e seus respectivos tempos de ociosidade são

mantidas na struct **CPU**. O contador de ociosidade da CPU é a variável **CPUs[i].tempo_desligado++**.

Por fim, o tick atual é derivado de uma variável de nome **int tick**, também armazenada em **EstadoSistema**. Sua exibição é feita pelo comando **printf("Tick atual: %d\n\n", atual_estado->tick)**.

Trecho:

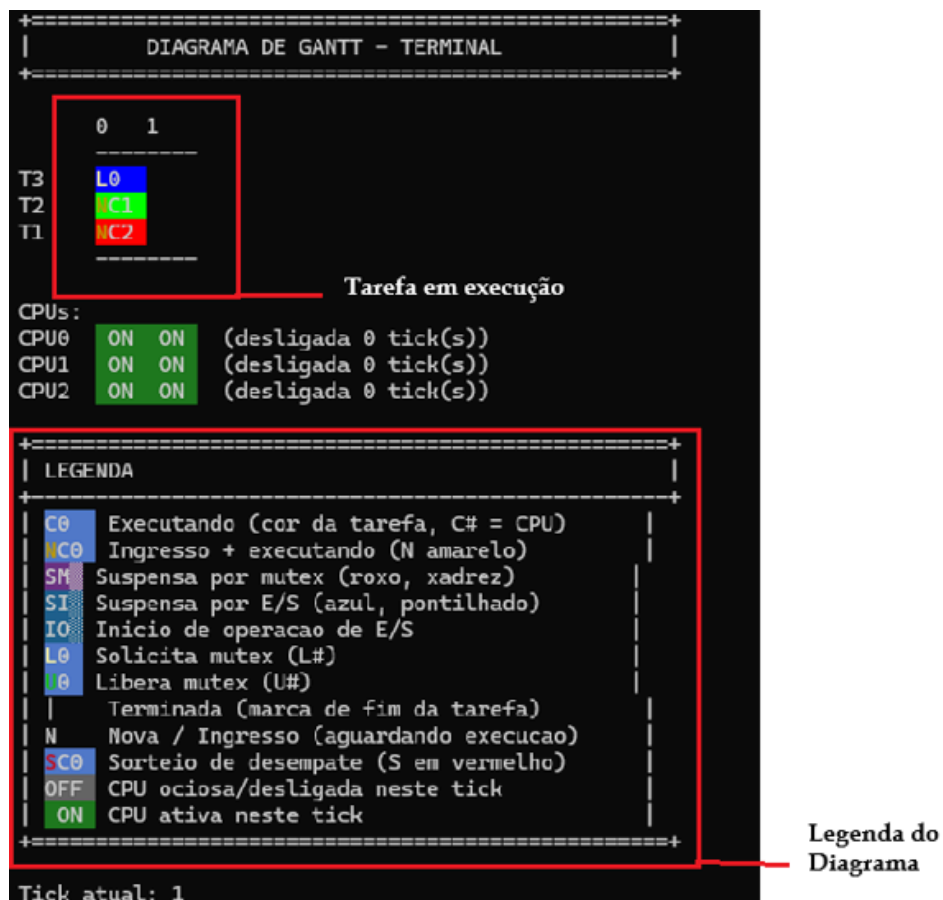
```
A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

>
```

Partindo propriamente para as opções das quais o usuário pode escolher para executar na simulação, temos:

1.3.1. Avançar (A)

Caso o usuário escolha avançar, o simulador executará um tick, indo diretamente para o seguinte. Isso ocorre através de uma chamada de função dentro da função responsável pela execução passo a passo.



Temos então o avanço para o tick seguinte da simulação, onde algumas situações referentes às tarefas foram atualizadas. Por exemplo, a tarefa T0 entrou na CPU 0 e está sendo executada. Essa função de avançar é responsável por chamar a função "**executar_tick()**" que, como o nome diz, realiza a execução de um tick. Nessa mesma função é chamada a "**simular_tick()**", responsável por modificar as tarefas e as CPUs, alterar estados e atualizar o diagrama de Gantt.

1.3.2. Retroceder (R)

Nessa opção, o simulador retorna para o tick imediatamente anterior, restaurando o estado que as tarefas e as CPUs se encontravam, incluindo o tempo restante para o término da tarefa, sua duração e sua representação gráfica no diagrama de Gantt. Como o tick, o retroceder deriva da struct **EstadoSistema**, que semelhantemente a uma lista, armazena "snapshots" do sistema, onde ponteiros são responsáveis por permitir o retrocesso (ponteiro para o snapshot anterior) ou o avanço (ponteiro para o snapshot seguinte). Isso é bem perceptível no comando: **atual_estado = atual_estado->anterior;**

1.3.3. Modificar (M)

Agora, caso o usuário digite "M", a opção escolhida pelo mesmo permite com que ele realize a mudança de estado das tarefas no tick atual. Para isso, ele é novamente direcionado para outra tela. Ainda na função "**execucaoPassoAPasso**", a opção de modificar o estado utiliza uma variável que armazena o estado escolhido e então atualiza o estado da tarefa. Além disso, ao optar pela modificação do estado, o simulador também, se necessário conforme o estado modificado, remove da fila; remove da CPU, recoloca na fila, encerra tarefa e atualiza contexto.

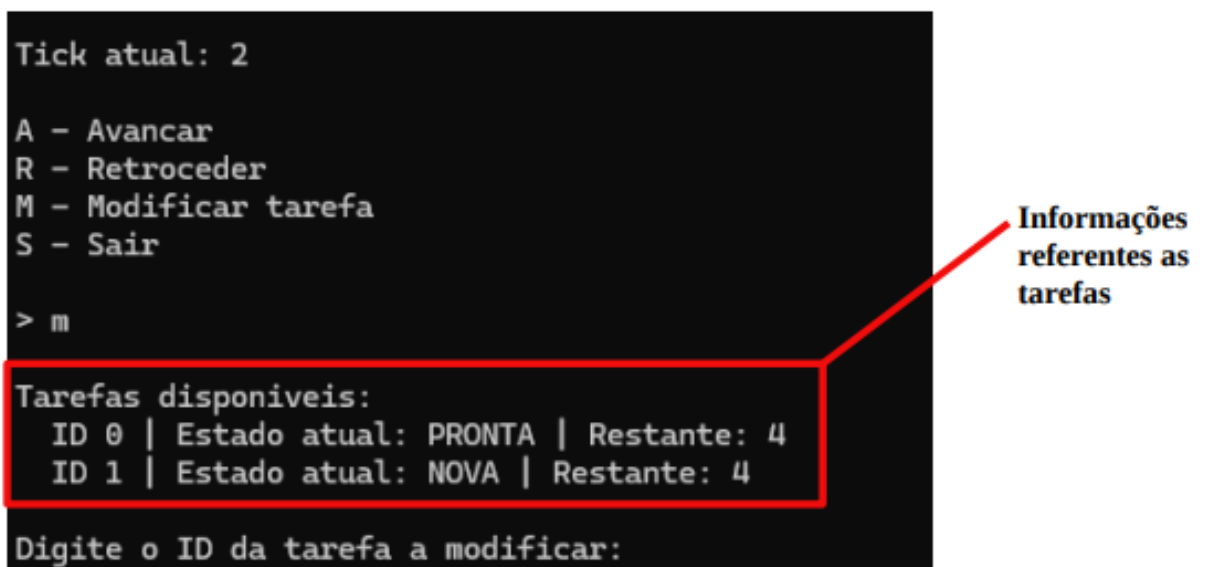
```
Tick atual: 2

A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

> m

Tarefas disponiveis:
  ID 0 | Estado atual: PRONTA | Restante: 4
  ID 1 | Estado atual: NOVA | Restante: 4

Digite o ID da tarefa a modificar:
```



Informações referentes as tarefas

Nessa tela, é apresentado as tarefas disponíveis na execução, bem como seus respectivos IDs, o estado no qual se encontram e o tempo restante de execução delas. Para avançar o usuário escolhe o id da tarefa que deseja modificar

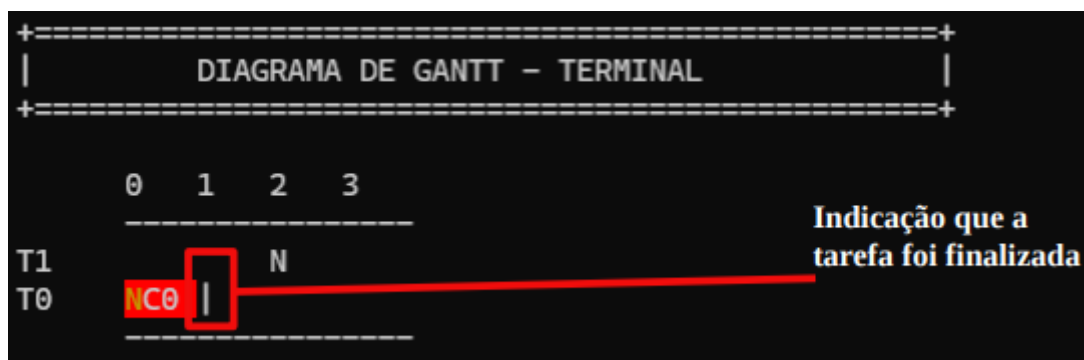
```
Digite o ID da tarefa a modificar: 0

Escolha o novo estado para a Tarefa 0:
1 - NOVA
2 - PRONTA
3 - EXECUTANDO
4 - TERMINADA
5 - SUSPENSA
Digite o numero do estado:
```

Assim, uma nova tela é apresentada para o usuário, agora apresentando as opções que se tem para modificar o estado da tarefa anteriormente selecionada. Assim, aquele que manipula a simulação precisa digitar o número referente a tarefa, como indicado no terminal. Estes números são os mesmos utilizados para definir internamente cada estado, portanto, caso haja a inserção de um número não presente nas opções, uma mensagem de erro é indicada e não é realizada nenhuma mudança.

```
! Estado invalido. Nenhuma alteracao realizada.
```

Ademais, caso seja selecionado um número válido e consequentemente um estado novo, a função passo a passo altera o estado da tarefa e realiza o tratamento da mesma, realizando as alterações necessárias para com que a mudança seja verdadeiramente aplicada em toda a tarefa, influenciando todos os seus atributos e sua representação gráfica, e não somente mude o valor interno do estado dela.



Neste exemplo, modifiquei o estado da tarefa T0 para terminada, então a tarefa é devidamente encerrada.

1.3.4. Examinar (E)

Ao digitar "E", o usuário acessa uma tela de inspeção, semelhante à visão de variáveis de um **debugger**, que apresenta, em forma de tabela, o estado detalhado de todas as tarefas no tick atual. Essa opção é útil para acompanhar de perto a evolução da simulação e entender o porquê de cada decisão de escalonamento.

```
=== Estado das tarefas no tick 1 ===
```

ID	Estado	Prio	Ingr	Dur	Rest	Qtm	CPU	Obs
T1	EXECUTANDO	5	0	8	7	99	2	
T2	EXECUTANDO	9	0	7	6	99	1	
T3	EXECUTANDO	9	0	5	4	99	0	

Pressione ENTER para voltar ao menu...|

Para cada tarefa, a tabela exibe as seguintes colunas:

- **ID**: identificador da tarefa;
- **Estado**: situação atual (NOVA, PRONTA, EXECUTANDO, TERMINADA ou SUSPENSA);
- **Prio (ou Prio/Din no PRIOPEnv)**: a prioridade estática e, no caso do PRIOPEnv, também a prioridade dinâmica resultante do envelhecimento;
- **Ingr**: instante de ingresso da tarefa;
- **Dur**: duração total;
- **Rest**: tempo restante até o término;
- **Qtm**: quantum restante;
- **CPU**: CPU em que a tarefa está executando (ou "-" se não estiver em nenhuma);
- **Obs**: observação sobre o motivo da suspensão, indicando se a tarefa está suspensa por mutex (e qual mutex) ou por E/S (com o tick em que a IRQ ocorrerá).

Ao final, o simulador aguarda o usuário pressionar **ENTER** para voltar ao menu, sem alterar o estado da simulação (é apenas uma consulta).

Essa tela é gerada dentro da função "**execucao_passo_a_passo()**", a partir dos dados da struct **TCB** (estado, prioridades, tempos e motivo de suspensão) e da struct **CPU** (para descobrir em qual processador cada tarefa está rodando).

1.3.5. Sair (S)

```
Tick atual: 1

A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

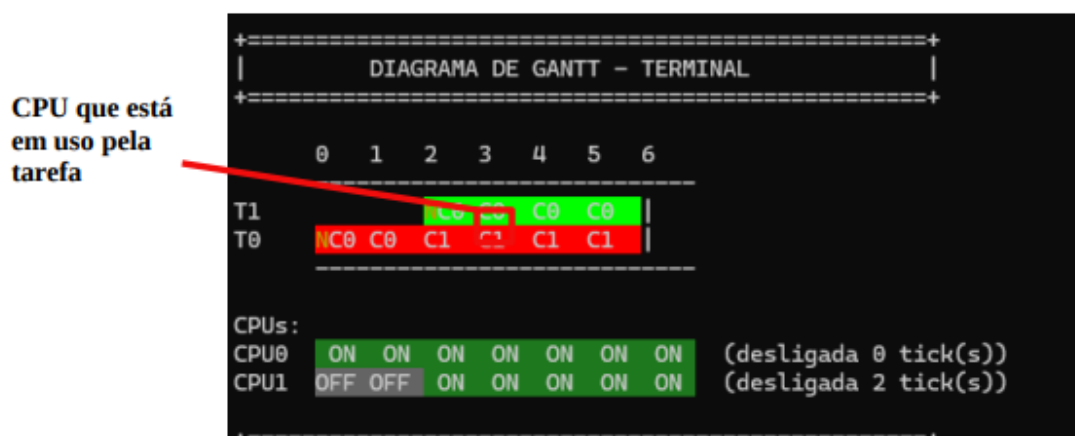
> S

Ociosidade por CPU:
  CPU 0: desligada por 0 tick(s)
  CPU 1: desligada por 1 tick(s)

Process returned 0 (0x0)   execution time : 1482.975 s
Press any key to continue.
```

Por fim, caso o usuário escolha “Sair”, a simulação é encerrada no tick atual e apresenta a ociosidade, ou seja, o tempo de desligamento total, que cada CPU teve durante a execução.

1.4. Execução Completa



Por outro lado, caso o usuário escolha realizar a execução completa(b), a função que é chamada na main é a referente a este modo de execução. Será apresentado no terminal somente a versão final do diagrama, não permitindo modificações ou retrocessos. Também é apresentada todo o uso das CPUs e o tempo de ociosidade total ao seu lado

1.5. Legenda do diagrama (atualizada para o Projeto B)

Logo abaixo do diagrama de Gantt, tanto no terminal quanto no arquivo .svg, é apresentada uma legenda que explica cada convenção visual utilizada na representação. No terminal, cada item da legenda é impresso com o mesmo esquema de cores ANSI das células do diagrama, de modo que o usuário vê um exemplo real de cada símbolo. Em relação ao Projeto A, a legenda foi ampliada para contemplar os novos recursos do Projeto B, os mutexes e as operações de entrada/saída. Os itens da legenda são:

LEGENDA	
C0	Executando (cor da tarefa, C# = CPU)
NC0	Ingresso + executando (N amarelo)
SM	Suspensa por mutex (roxo, xadrez)
SI	Suspensa por E/S (azul, pontilhado)
IO	Início de operação de E/S
L0	Solicita mutex (L#)
U0	Libera mutex (U#)
	Terminada (marca de fim da tarefa)
N	Nova / Ingresso (aguardando execução)
SC0	Sorteio de desempate (S em vermelho)
OFF	CPU ociosa/desligada neste tick
ON	CPU ativa neste tick

- **C0** — tarefa executando, exibida na cor da própria tarefa; o número (C#) indica a CPU que está executando aquela tarefa;
- **NC0** — ingresso + execução: a tarefa acabou de chegar e já entrou em execução; o "N" em amarelo marca a chegada;
- **SM** : tarefa suspensa por mutex, em fundo roxo com padrão quadriculado (xadrez);
- **SI**: tarefa suspensa por E/S, em fundo azul com padrão pontilhado;
- **IO**: início de uma operação de E/S (marca o tick em que a E/S começa);
- **L#**: a tarefa solicita/trava um mutex (*lock*), com o número do mutex;
- **U#**: a tarefa libera um mutex (*unlock*), com o número do mutex;
- **|**: marca de término da tarefa (fim da execução);
- **N**: tarefa nova, que já ingressou mas ainda aguarda execução;
- **SC0**: tarefa escolhida por sorteio de desempate (o "S" em vermelho);
- **OFF**: CPU ociosa/desligada naquele tick;
- **ON**: CPU ativa naquele tick.

Os três itens que caracterizam o Projeto B são o SM e o SI, que diferenciam visualmente os dois motivos de suspensão (por mutex e por E/S), enquanto no Projeto A a tarefa suspensa era representada de forma única (em preto), e os marcadores L#/U# e IO, que sinalizam os eventos de mutex e de E/S ao longo da execução. Essa legenda é impressa pela função `"imprimir_gantt_terminal()"`, e a versão correspondente no arquivo .svg utiliza os mesmos padrões (quadriculado para mutex e listras diagonais para E/S).

2. Diagrama .svg

2.1. Geração do arquivo .svg

Em ambos modos de execução, é criado um arquivo **.svg** no mesmo diretório que se encontra o simulador. Este arquivo apresenta o diagrama de Gantt da execução. No caso da execução passo a passo, é gerado um diagrama para cada tick executado, seja um avanço ou um retrocesso, e as modificações realizadas são igualmente apresentadas, tendo em seu nome “*modificado*” .

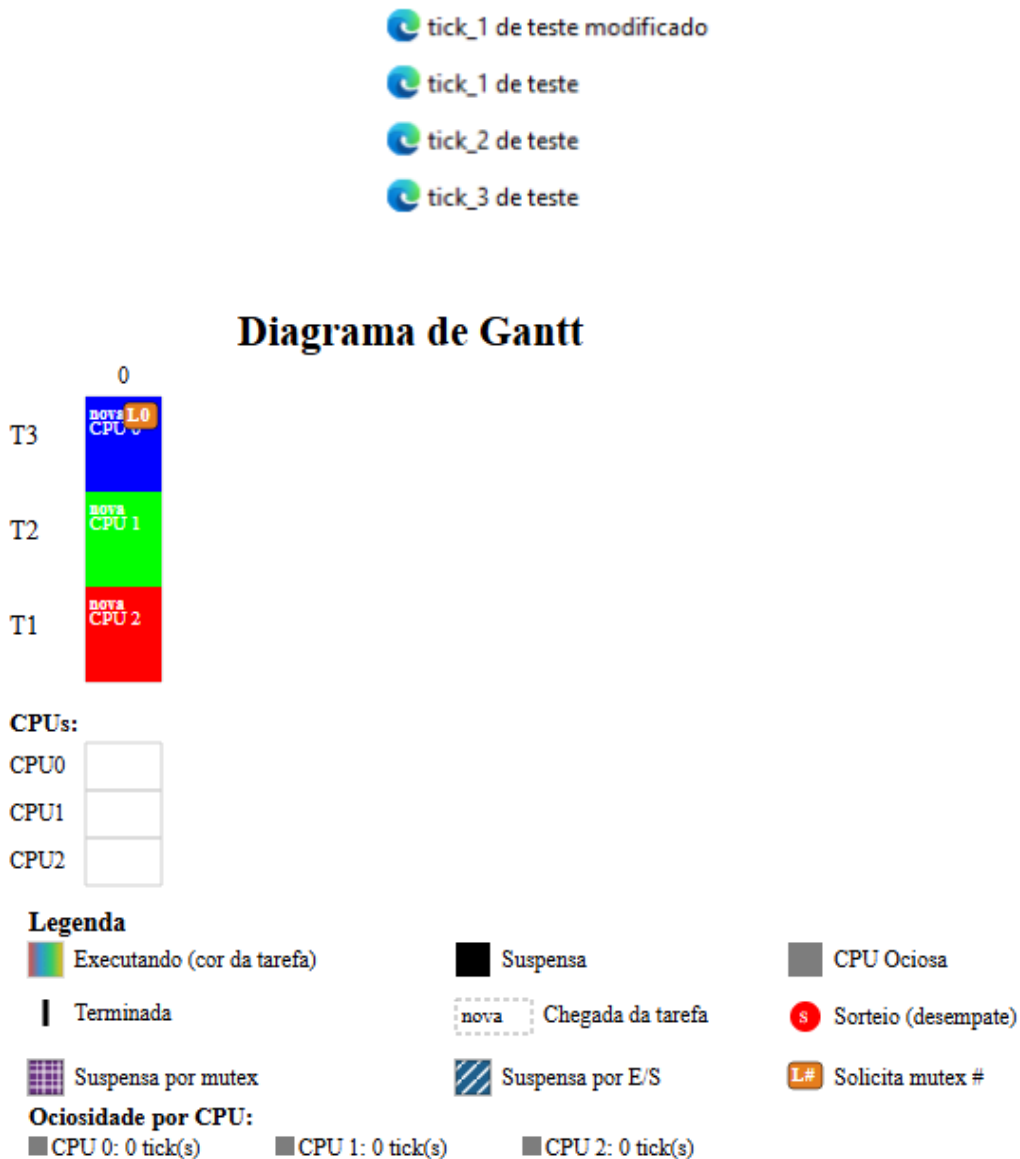
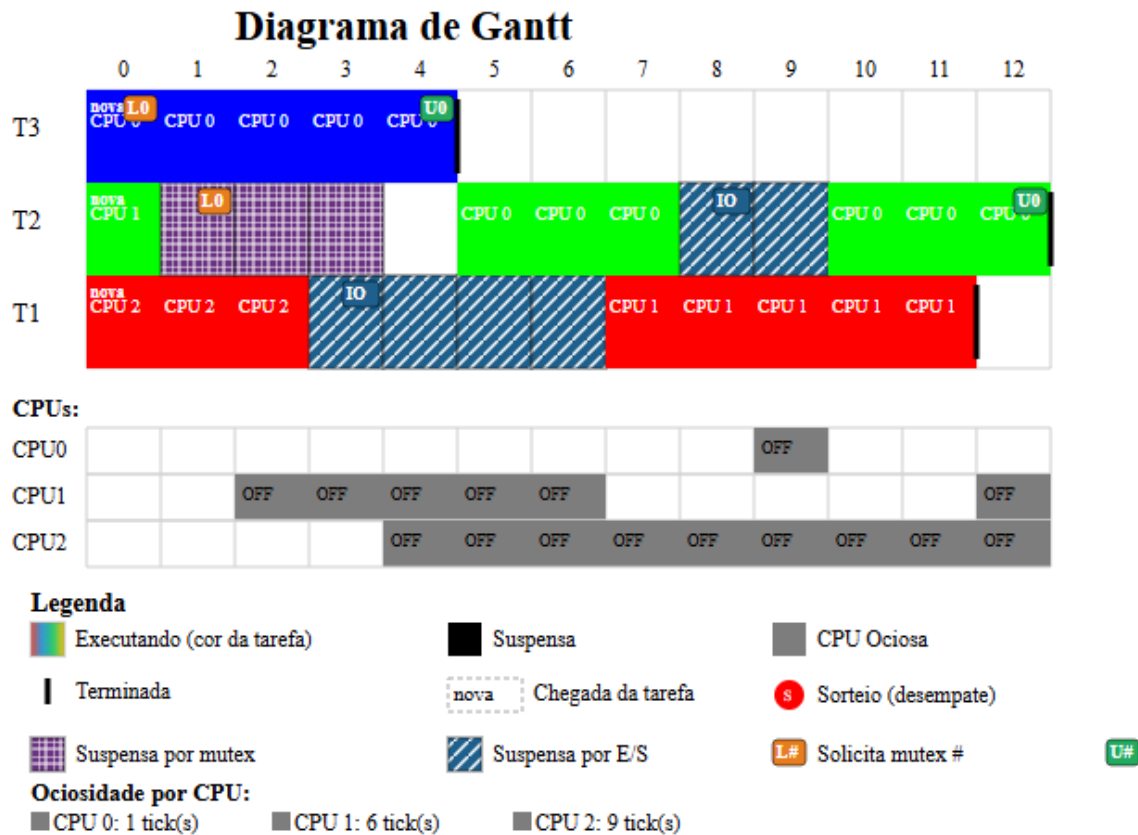


Diagrama de Gantt da modificação apresentada em 1.3.3

Na execução completa, é gerado apenas o diagrama de Gantt final, onde já foi realizado toda a simulação.



Nesse diagrama de Gantt, é apresentado cada CPU que está sendo utilizada por determinada tarefa em determinado tick, bem como uma legenda que indica cada um dos casos que a tarefa pode se encontrar. Esses diagramas são gerados pela função “*criaSVG()*”, e por padrão, o arquivo vem com o nome no formato ***config.nome_arquivo***.

3. Recursos do Projeto B

3.1. Cabeçalho PRIOPEnv (algoritmo;quantum;cpus;alpha)

O Projeto B introduz um novo algoritmo de escalonamento: o PRIOPEnv (Prioridade Preemptivo com Envelhecimento). Ele funciona como o PRIOP (escolhe sempre a tarefa de maior prioridade e é preemptivo), mas acrescenta o conceito de envelhecimento (*aging*): a prioridade das tarefas que ficam muito tempo aguardando é aumentada gradualmente, evitando que tarefas de baixa prioridade nunca sejam executadas (*starvation*).

Para suportar esse recurso, a primeira linha do arquivo de configuração ganhou um quarto campo, o alpha, que é o fator de envelhecimento:

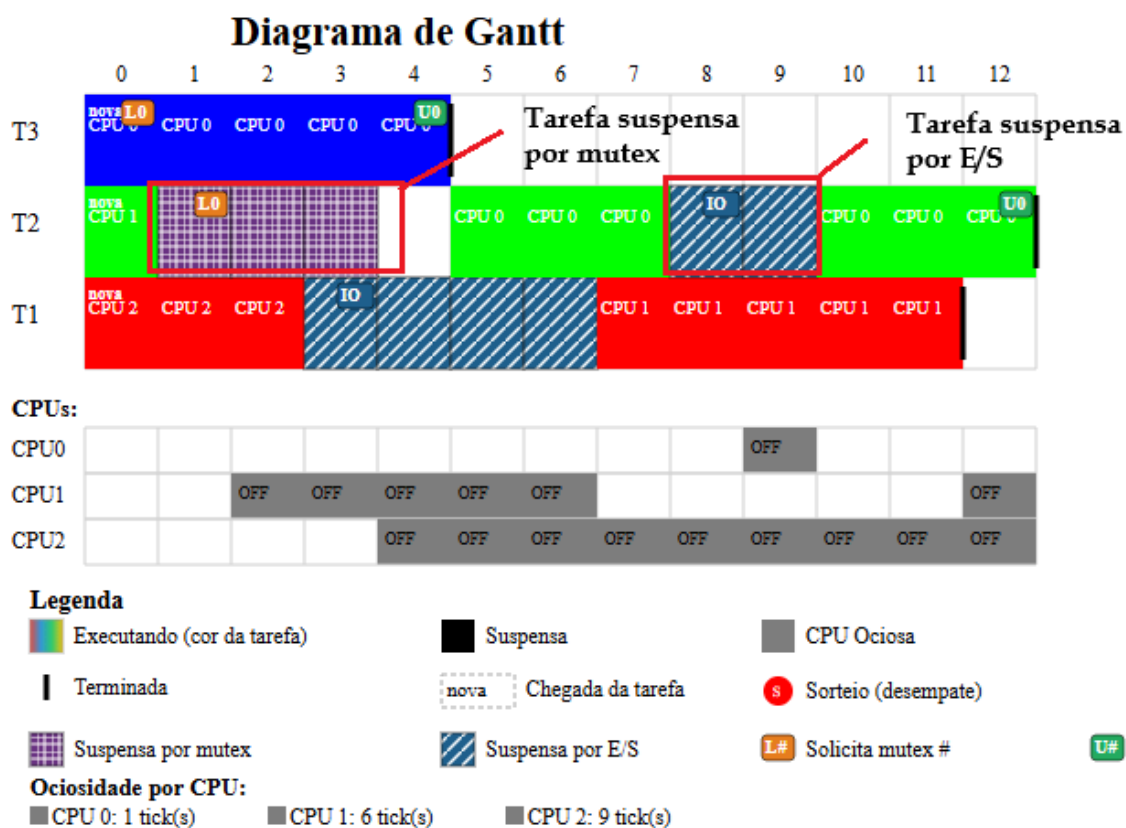
PRIOPEnv;quantum;qtde_cpus;alpha

3.2. Mutex — Lock e Unlock (ML/MU)

O Projeto B adiciona suporte a mutexes para exclusão mútua. Na lista de eventos, **MLxx:nn** trava (*lock*) o mutex **xx** no instante **nn** da tarefa, e **MUxx:nn** o libera (*unlock*).

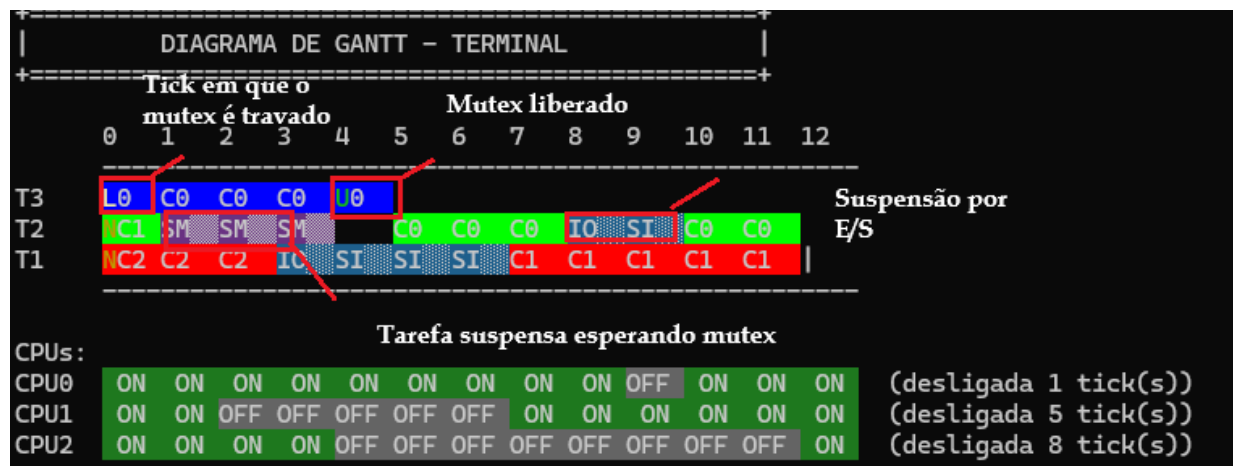
Quando uma tarefa executa um **ML**, se o mutex estiver livre ela o adquire e continua; se estiver ocupado por outra, ela é suspensão por mutex e liberada da CPU. Ao executar o **MU**, o mutex é liberado e a próxima tarefa que esperava por ele é acordada, a escolha segue a ordem *FIFO* (pelo *tick_bloqueio*).

No terminal, o *lock* aparece como **L#** e o *unlock* como **U#**; a tarefa que espera o mutex fica em **SM** (roxo, xadrez). No **SVG**, os mesmos eventos aparecem sobre o bloco e a suspensão usa o padrão quadriculado.

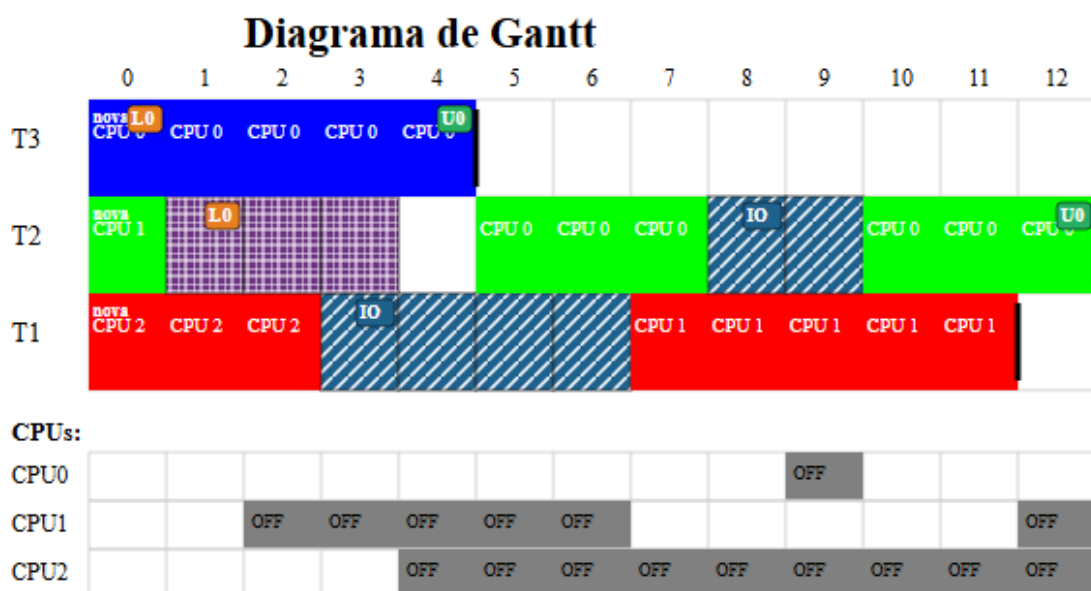


3.2.1. Terminal (L, U e SM)

No terminal, os eventos de mutex aparecem sobre o bloco da tarefa: **L#** no *tick* em que ela trava o *mutex* e **U#** no *tick* em que o libera (# = número do mutex). A tarefa que tenta travar um *mutex* já ocupado fica em **SM**, com fundo roxo e textura xadrez, até conseguir o *mutex*.



3.2.2. SVG (seção crítica e suspensão quadriculada)



No arquivo **.svg**, a seção crítica é destacada sobre o bloco da tarefa: os marcadores **L#** e **U#** indicam onde o *mutex* é travado e liberado, delimitando o trecho em que a tarefa detém o recurso. A tarefa suspensa esperando o mutex é desenhada com o padrão quadriculado sobre fundo roxo, diferenciando-a visualmente da suspensão por E/S.

3.3. Operações de E/S (IO)

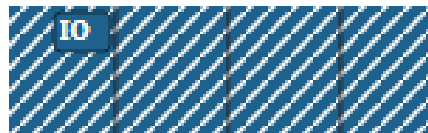
O Projeto B trata operações de entrada/saída. Na lista de eventos, **IO:xx-yy** inicia uma E/S no instante **xx** da tarefa, com duração **yy** ticks. Ao atingir o instante **xx**, a tarefa é suspensa por E/S e liberada da CPU; quando a operação termina, a IRQ ocorre no *tick* seguinte e a tarefa volta para a fila de prontas.

3.3.1. Terminal (IO e SI)

No terminal, o tick em que a E/S começa é marcado com **IO**. Enquanto a operação está em andamento, a tarefa fica em **SI**, com fundo azul e textura pontilhada, até a IRQ acordá-la.



3.3.2. SVG (suspensão diagonal)



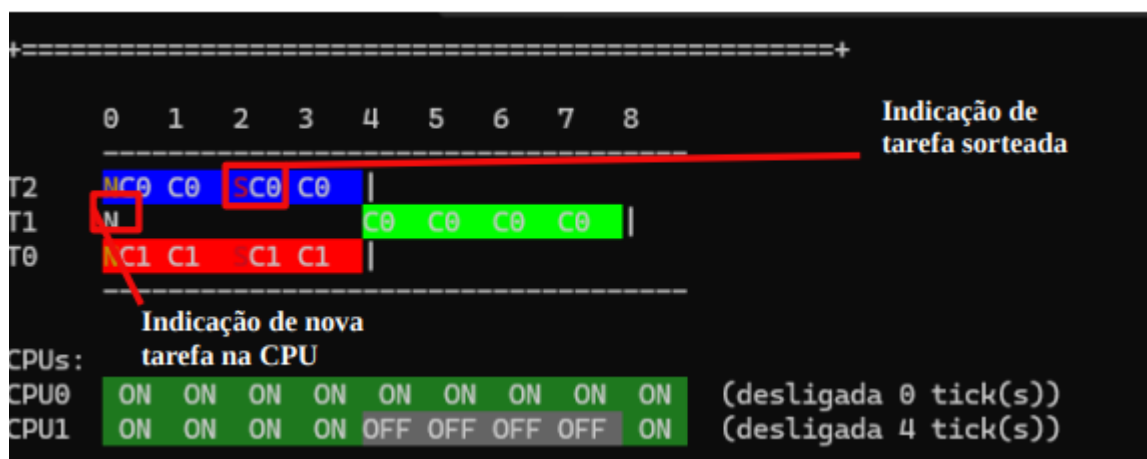
4. Casos especiais

4.1. Tarefa sorteada

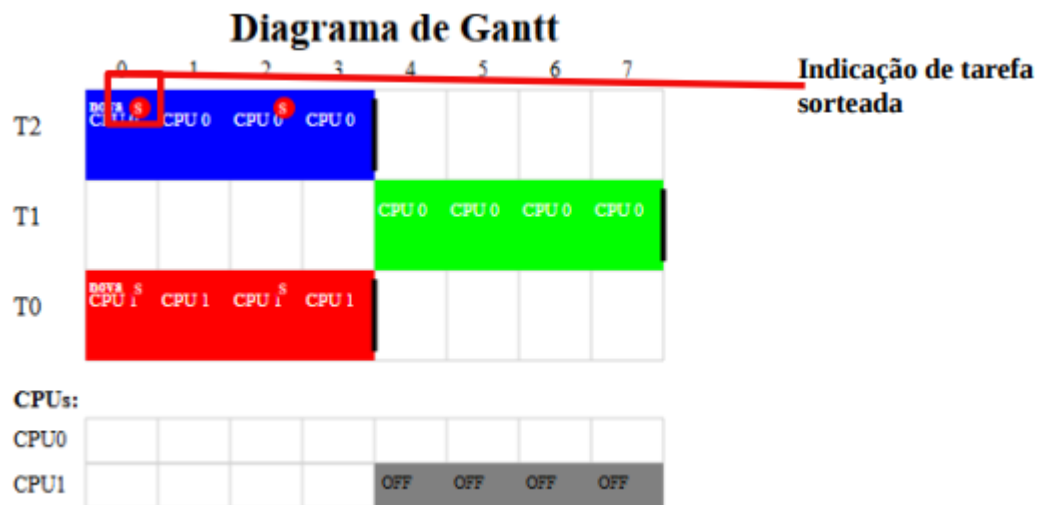
Como definido em ambas opções de escalonamento, temos como último critério de desempate um sorteio para saber qual tarefa será a próxima a utilizar a CPU em questão. Dessa forma, tem-se um elemento visual para indicar que ela foi escolhida via sorteio, como indica as legendas do terminal e do diagrama de Gantt no formato **.svg**:

4.1.1. Terminal (S vermelho)

No terminal, temos um indicador S ao lado da CPU que está em uso para representar que o critério de desempate utilizado para a escolha da tarefa a executar foi um sorteio.



4.1.2. SVG (S em círculo vermelho)



No diagrama de Gantt gerado no formato **.svg**, o indicador da utilização do sorteio como critério de desempate é apresentado como um pequeno S dentro de um círculo vermelho na tarefa sorteada. Essa sinalização de tarefa sorteada é feita utilizando o campo int **sorteio** da struct **DiagramaGantt**. De maneira geral, os dados essenciais para o funcionamento da simulação são obtidos de “**ConfigSistema lerConfiguracao()**”, responsável pela leitura dessas informações no arquivo **.txt** disponibilizado.

5. Conclusão

O desenvolvimento do Projeto B consolidou e ampliou o simulador de sistema operacional multitarefa construído no Projeto A. Além dos algoritmos SRTF e PRIOP já existentes, foi implementado o **PRIOPenv**, que introduz o envelhecimento (*aging*) por meio de uma prioridade dinâmica ajustada pelo fator alpha, evitando a inanição de tarefas de baixa prioridade.

Os principais recursos acrescentados foram o suporte a **mutexes** (com as operações de *lock* e *unlock* e o bloqueio de tarefas que disputam um recurso), o tratamento de **operações de entrada/saída** (com suspensão da tarefa e retorno via IRQ) e a **diferenciação visual dos motivos de suspensão**, mutex e E/S passaram a ter cores e padrões próprios, tanto no terminal quanto no diagrama **.svg**, superando a representação única do Projeto A. Somam-se a isso melhorias de robustez, como a remoção dos limites fixos (número de tarefas, eventos, mutexes e ticks passaram a ser dimensionados dinamicamente) e a leitura de arquivos por caminho completo.

Assim como no Projeto A, o simulador é resultado da interação organizada entre diversas funções e estruturas de dados bem definidas, como as structs **TCB**, **CPU**, **EstadoSistema** e **DiagramaGantt**, cuja comunicação clara é essencial para o funcionamento correto da

simulação. Para manter o código legível e acessível a novos programadores, foram mantidos comentários explicativos ao longo de toda a implementação. Dessa forma, o Projeto B cumpre os requisitos propostos e representa um exemplo prático e didático dos mecanismos de escalonamento, sincronização e gerenciamento de tarefas de um sistema operacional multitarefa.